

FORMULARIO DE COMBATE: CC1002 – CONTROL 2

Felipe Colli Olea – FCFM Universidad de Chile – Primavera 2026

1. LA SAGRADA RECETA DE DISEÑO (PUNTAJE FIJO)

Obligatorio en CADA función (hasta 1.5 - 2.0 pts por pregunta):

```
# nombreFuncion: tipo1 tipo2 -> tipoRetorno
# Proposito: Breve descripcion de que calcula
# Ejemplo: nombreFuncion(arg1, arg2) devuelve resultado
def nombreFuncion(param1, param2):
    return resultado
```

```
# Tests (Al menos 1 o 2 tests por funcion):
assert nombreFuncion(val1, val2) == esperado
assert not funcionBooleana(casoFalso)
```

Contratos válidos en C2:

- **Estructuras y listas:** lista(int), lista(str), lista(any), lista(Carta), AB, ABB, nodoAG.
- **Funciones de orden superior:** (any ->bool), (any ->any), (any any ->any).
- **Básicos:** int, float, num, str, bool, None (procedimientos).

2. DATOS COMPUESTOS (IMPORT ESTRUCTURA)

Definición e Instanciación (INMUTABLES):

```
import estructura
# Registro: campo1(tipo1) campo2(tipo2)
estructura.crear('Registro', 'campo1 campo2')
r = Registro(val1, val2)
# Acceso a campos:
x = r.campo1; y = r.campo2
```

REGLA DE ORO: Los structs creados con `estructura.crear` **NO SON MUTABLES** (`r.campo1 = x` lanza `AttributeError`). Para “modificar”, se retorna una nueva instancia: `Registro(nuevo, r.campo2)`.

3. LISTAS RECURSIVAS NO MUTABLES (LISTA.PY)

Primitivas Fundamentales (from lista import *):

- `listaVacía = None` (Identificador para lista vacía).
- `lista(valor, resto)` o `crearLista(valor, resto)`: Crea nodo.
- `cabeza(L)`: Retorna valor en la cabeza (`L.valor`).
- `cola(L)`: Retorna resto de la lista (`L.siguiente`).
- `vacía(L)`: Retorna `True` si `L == listaVacía`.

Plantillas Estándar de Listas

1. Largo / Conteo:

```
def largo(L):
    if vacía(L): return 0
    return 1 + largo(cola(L))
```

2. Búsqueda de elemento (retorna dato o None):

```
def buscar(L, clave):
    if vacía(L): return None
    if cabeza(L).id == clave: return cabeza(L)
    return buscar(cola(L), clave)
```

3. Unión de dos listas (preserva orden):

```
def unionListas(L1, L2):
    if vacía(L1): return L2
    return lista(cabeza(L1), unionListas(cola(L1), L2))
```

4. Invertir lista:

```
def invertir(L, acc=listaVacía):
    if vacía(L): return acc
    return invertir(cola(L), lista(cabeza(L), acc))
```

5. Procesamiento de a Pares / Consecutivos (C2 2025):

```
def pares(L):
    if vacía(L) or vacía(cola(L)): return listaVacía
    c_act = cabeza(L); c_sig = cabeza(cola(L))
    if c_act.valor == c_sig.valor:
        resto = pares(cola(cola(L)))
        return lista(c_act, lista(c_sig, resto))
    return pares(cola(L))
```

4. ÁRBOLES BINARIOS (AB)

Definición e Identificador Vacío:

```
# AB: valor(any) izq(AB) der(AB)
estructura.crear('AB', 'valor izq der')
ABVacío = None
# vacío(A): A == ABVacío (o A == None)
```

Condición de Hoja: `A.izq == None and A.der == None`.
Operaciones Clave en AB

1. Contar Nodos:

```
def contar(A):
    if A == None: return 0
    return 1 + contar(A.izq) + contar(A.der)
```

2. Contar Hojas:

```
def hojas(A):
    if A == None: return 0
    if A.izq == None and A.der == None: return 1
    return hojas(A.izq) + hojas(A.der)
```

3. Altura de un Árbol:

```
def altura(A):
    if A == None: return 0
    return 1 + max(altura(A.izq), altura(A.der))
```

4. Recorridos a Lista:

- **Preorden (R-I-D):** lista(A.valor, unionListas(pre(A.izq), pre(A.der)))
- **Inorden (I-R-D):** unionListas(inord(A.izq), lista(A.valor, inord(A.der)))
- **Postorden (I-D-R):** unionListas(post(A.izq), unionListas(post(A.der), lista(A.valor, listaVacía)))

5. Árboles de Torneo / Relación Padre-Hijo (C2 2024):

```
def validarTorneo(A):
    if A == None: return False
    if A.izq == None and A.der == None: return True
    ambos_ok = validarTorneo(A.izq) and validarTorneo(A.der)
    gana_uno = (A.valor == A.izq.valor or A.valor == A.der.valor)
    return ambos_ok and gana_uno
```

5. ÁRBOLES BINARIOS DE BÚSQUEDA (ABB)

Propiedad ABB: $\forall$ nodo X : Subárbol Izq $< X <$ Subárbol Der.

1. Búsqueda 0(h):

```
def buscarABB(A, x):
    if A == None: return None
    if A.valor == x: return A
    elif x < A.valor: return buscarABB(A.izq, x)
    else: return buscarABB(A.der, x)
```

2. Camino hacia un valor (Lab 7):

```
def camino(A, x):
    if A == None: return None
    if A.valor == x: return lista(x, None)
    elif x < A.valor:
        return lista(A.valor, camino(A.izq, x))
    else:
        return lista(A.valor, camino(A.der, x))
```

3. Insercion no mutable en ABB:

```
def insertarABB(A, x):
    if A == None: return AB(x, None, None)
    if x < A.valor:
        return AB(A.valor, insertarABB(A.izq, x), A.der)
    elif x > A.valor:
        return AB(A.valor, A.izq, insertarABB(A.der, x))
    return A
```

4. Minimo y Maximo en ABB:

```
def minimoABB(A):
    assert A != None
    if A.izq == None: return A.valor
    return minimoABB(A.izq)
```

```
def maximoABB(A):
```

```

assert A != None
if A.der == None: return A.valor
return maximoABB(A.der)

```

5. Verificar si un AB es ABB valido:

```

def esABB(A, mini=-float('inf'), maxi=float('inf')):
    if A == None: return True
    if not (mini < A.valor < maxi): return False
    return esABB(A.izq, mini, A.valor) and \
           esABB(A.der, A.valor, maxi)

```

6. ABSTRACCIÓN FUNCIONAL Y ORDEN SUPERIOR

Funciones Anónimas (lambda)

Sintaxis: lambda arg1, arg2, ...: expresion

```

lambda x: x + 1           # Sumar 1
lambda c: c.pinta == 'C'  # Predicado booleano
lambda x, y: x + y        # Acumulador binario

```

Las 3 Funciones Abstractas Sagradas

1. Mapa (mapa): Transforma cada elemento de la lista:

```

# mapa: (X -> Y) lista(X) -> lista(Y)
def mapa(f, unaLista):
    if vacia(unaLista): return listaVacia
    return lista(f(cabeza(unaLista)), mapa(f, cola(unaLista)))

```

2. Filtro (filtro): Filtra elementos que cumplen condición:

```

# filtro: (X -> bool) lista(X) -> lista(X)
def filtro(operador, unaLista):
    if vacia(unaLista): return listaVacia
    if operador(cabeza(unaLista)):
        return lista(cabeza(unaLista),
                     filtro(operador, cola(unaLista)))
    return filtro(operador, cola(unaLista))

```

3. Reductor (fold): Reduce la lista a un único valor acumulado:

```

# fold: (Y X -> Y) Y lista(X) -> Y
def fold(f, init, unaLista):
    if vacia(unaLista): return init
    return fold(f, f(init, cabeza(unaLista)), cola(unaLista))

```

Equivalencias clásicas con fold:

- suma = fold(lambda acc, x: acc + x, 0, L)
- prod = fold(lambda acc, x: acc * x, 1, L)
- largo = fold(lambda acc, x: acc + 1, 0, L)
- invertir = fold(lambda acc, x: lista(x, acc), listaVacia, L)

Mapa sobre Árboles Binarios (mapaAB - Lab 6)

mapaAB: (X -> Y) AB(X) -> AB(Y)

```

def mapaAB(f, unAB):
    if unAB == None: return None
    return AB(f(unAB.valor),
             mapaAB(f, unAB.izq),
             mapaAB(f, unAB.der))

```

Patrón Pipeline (Filtro + Mapa Combinados)

```

# Ej C2 2024: buscar por ids y filtrar None
resultado = mapa(lambda id: buscarPersona(LP, id), Lids)
return filtro(lambda p: p != None, resultado)

```

7. ÁRBOLES GENEALÓGICOS / N-ARIOS

```

# nodoAG: padre(nodoAG) madre(nodoAG)
#         nombre(str) nac(int) ojos(str)
estructura.crear('nodoAG', 'padre madre nombre nac ojos')
nodoAGVacio = None

```

Buscar si algun ancestro cumple condicion:

```

def ancestroOjosAzules(unAG):
    if unAG == None: return False
    return (unAG.ojos == 'azules') or \
           ancestroOjosAzules(unAG.padre) or \
           ancestroOjosAzules(unAG.madre)

```

8. ERRORES FATALES Y TRAMPAS TÍPICAS EN C2

- **AttributeError: 'NoneType' object has no attribute 'x':** Ocurre al hacer A.izq.valor o cabeza(cola(L)) sin verificar antes si el subárbol o cola es None. ¡SIEMPRE caso base antes!
- **Inversión en ABB:** En un ABB:

- val <A.valor → buscar en **IZQUIERDA** (A.izq).

- val >A.valor → buscar en **DERECHA** (A.der).

- **Preservar el Orden en Listas:** lista(cabeza(L), llamadaRec(cola(L))) preserva orden. Si pones el nodo nuevo como acumulador antes de la recursión, la lista queda al revés.

- **Cuidado con Listas de 1 elemento:** Si tu función accede a cabeza(cola(L)), tus casos base deben ser dos: lista vacía y lista de 1 elemento (cola(L) == listaVacia).

- **No usar bucles prohibidos:** En Unidades I y II NO se permite for, while, ni listas []. Todo debe resolverse con recursión pura sobre lista.py y AB.